

OS Lab Assignment: System Calls and Process Creation

Name: Hitesh sangwan **Roll Number:** 2401730218

1. Objective & Overview

This lab assignment demonstrates core Operating System concept implementation in Python via low-level system call interfaces provided by the `os` module. Key system level mechanisms covered include process creation, binary execution, direct file descriptor manipulation, device and virtual file system interactions, and OS-level error handling.

2. Key System Concepts Addressed

- **Process Creation & Management (fork / exec):** Demonstrates creating child processes using `os.fork()`, replacing execution context using `os.execvp()`, and process synchronization with `os.wait()`.
- **File Descriptor Operations:** Uses direct I/O system calls (`os.open`, `os.write`, `os.read`, and `os.close`) rather than high-level language wrappers to interact directly with the operating system kernel file table.
- **Device & Virtual File System Access:** Interacts with special device nodes like `/dev/null` and reads kernel status directly from `/proc/version`.
- **Error Handling:** Handles system-level errors (such as `OSError`) gracefully when invalid paths or illegal operations are attempted.

3. Python Implementation Code

```
import os

TEST_FILE = "test_file.txt"

def process_creation_demo():
    pid = os.fork()
```

```

    if pid == 0:
        print(f"Child process -> PID: {os.getpid()}, Parent PID:
{os.getppid()}")
        os.execlp("echo", "echo", "Hello from exec, running inside the
child process")
    else:
        os.wait()
        print(f"Parent process -> PID: {os.getpid()}, Parent PID:
{os.getppid()}")

def file_operations_demo():
    fd = os.open(TEST_FILE, os.O_CREAT | os.O_WRONLY | os.O_TRUNC)
    os.write(fd, b"Hello from the OS lab test file\n")
    os.close(fd)

    fd = os.open(TEST_FILE, os.O_RDONLY)
    data = os.read(fd, 100)
    os.close(fd)

    print(f"Wrote, read and closed '{TEST_FILE}':
{data.decode().strip()}")
    os.remove(TEST_FILE)

def device_interface_demo():
    fd = os.open("/dev/null", os.O_WRONLY)
    os.write(fd, b"this goes nowhere")
    os.close(fd)
    print("Wrote to /dev/null successfully (data is discarded, no error)")

    with open("/proc/version", "r") as f:
        print("Read /proc/version:", f.readline().strip())

def error_handling_demo():
    try:
        os.open("/no/such/path/file.txt", os.O_RDONLY)
    except OSError as e:
        print(f"Handled expected error opening invalid path: {e}")

```



```

print("PROCESS CREATION (fork + exec)")
process_creation_demo()

print("\nFILE OPERATIONS")
file_operations_demo()

print("\nDEVICE AND PROC INTERFACE")
device_interface_demo()

print("\nERROR HANDLING")
error_handling_demo()

print("\nSUMMARY")
print("Process creation, wait and exec: done")
print("File create/write/read/close: done")
print("Device (/dev/null) and /proc interface access: done")
print("Invalid path handled with a clear error message: done")

```

4. Execution Output & Evidence

The screenshot below displays the console output verifying the successful execution of process creation (fork + exec), low-level file I/O operations, device interfaces, and exception handling.

```

(scienciac@ScienciacDev11):[~/OS-Second-Program]
$ python -u "/home/scienciac/OS-Second-Program/systemcall.py"
PROCESS CREATION (fork + exec)
Child process -> PID: 4111, Parent PID: 4110
Hello from exec, running inside the child process
Parent process -> PID: 4110, Parent PID: 1759

FILE OPERATIONS
Wrote, read and closed 'test_file.txt': Hello from the OS lab test file

DEVICE AND PROC INTERFACE
Wrote to /dev/null successfully (data is discarded, no error)
Read /proc/version: Linux version 6.6.87.2-microsoft-standard-ls2 (root@439a258ad544) (gcc (GCC) 11.2.0, GNU ld (GNU Binutils) 2.37) #1 SMP PREEMPT_DYNAMIC Thu Jun  5 18:30:46 UTC 2025

ERROR HANDLING
Handled expected error opening invalid path: [Errno 2] No such file or directory: '/no/such/path/file.txt'

SUMMARY
Process creation, wait and exec: done
File create/write/read/close: done
Device (/dev/null) and /proc interface access: done
Invalid path handled with a clear error message: done

```